database-physical-design

Documentación del Diseño Físico de Base de Datos - Sistema ZWL

1. Introducción

El diseño físico define la implementación del esquema lógico en el sistema gestor de bases de datos (MySQL 8.0+, motor InnoDB), especificando la organización de archivos, estructuras de almacenamiento, índices físicos, parámetros de configuración y políticas de respaldo. Este documento se enfoca en la implementación técnica a nivel de disco para el sistema ZWL.

---

2. Configuración del Motor de Almacenamiento

Todas las tablas de la base de datos `zwl` utilizan el motor **InnoDB**, seleccionado por:

- Soporte nativo para transacciones ACID
- Integridad referencial (claves foráneas)
- Bloqueo a nivel de fila (mejor concurrencia)
- Recuperación ante fallos mediante redo/undo logs

---

3. Organización Física de Archivos

3.1 Estructura de Tablespaces

InnoDB utiliza una configuración **file-per-table** (habilitada por defecto en MySQL 8.0), donde cada tabla tiene su propio archivo de tablespace:

- Formato: `nombre_tabla.ibd`
- Ubicación predeterminada (Windows): `C:\ProgramData\MySQL\MySQL Server 8.0\data\zwl\`
- El usuario está en entorno `win32`, por lo que los archivos se almacenan con rutas de estilo Windows.

3.2 Archivos del Sistema InnoDB

Archivo	Descripción	Ubicación
`ibdata1`	System tablespace (datos de diccionario, undo logs)	Directorio de datos de MySQL
`ib_logfile0`, `ib_logfile1`	Redo logs (registro de cambios para recuperación)	Directorio de datos de MySQL
`ibtmp1`	Temporary tablespace	Directorio de datos de MySQL

3.3 Archivos por Tabla (Ejemplo para `zwl`)

```
C:\ProgramData\MySQL\MySQL Server 8.0\data\zwl\
├── usuarios.ibd
├── productos.ibd
├── ventas.ibd
├── detalle_ventas.ibd
├── proveedores.ibd
├── solicitudes.ibd
├── activos.ibd
├── estaciones_cyber.ibd
├── sesiones_cyber.ibd
├── movimientos_stock.ibd
└── db.opt (obsoleto en MySQL 8.0, metadatos en diccionario de datos)
```

---

4. Almacenamiento Físico de Tipos de Datos

InnoDB almacena los datos con los siguientes tamaños fijos/variables:

Tipo de Dato	Tamaño de Almacenamiento	Notas Físicas
`INT`	4 bytes	Entero con signo, almacenado en formato binario little-endian
`DECIMAL(10,2)`	Variable (máx 5 bytes)	Almacenado como bytes binarios, 2 dígitos por byte
`VARCHAR(n)`	1-2 bytes de longitud + n bytes (utf8mb4: 1-4 bytes por carácter)	Longitud indica el número de bytes usados
`ENUM('a','b','c')`	1-2 bytes	Almacena un entero que mapea a la posición del valor en la lista
`DATE`	3 bytes	Formato: `YYYYMMDD`

Tipo de Dato	Tamaño de Almacenamiento	Notas Físicas
`DATETIME`	5 bytes	Formato: `YYYYMMDDHHMMSS` + microsegundos opcionales
`TIMESTAMP`	4 bytes	Almacena segundos desde la época Unix (1970-01-01)

---

5. Estructura Física de Índices

InnoDB utiliza árboles **B+ Tree** para todos los índices:

5.1 Índice Clustered (Primario)

- La clave primaria de cada tabla es un índice clustered: los datos de la fila se almacenan físicamente junto a las claves del índice.
- Ejemplo: Para la tabla `usuarios`, el archivo `usuarios.ibd` almacena el árbol B+ de la PK `id`, con las columnas `nombre`, `email`, etc. integradas en las hojas del árbol.

5.2 Índices Secundarios (No Clustered)

- Almacenan solo los valores de las columnas del índice y la clave primaria de la fila correspondiente.
- Para acceder a los datos completos, InnoDB usa la PK almacenada en el índice secundario para buscar en el índice clustered.
- Ejemplo: El índice `idx_ventas_fecha` en la tabla `ventas` almacena pares `(fecha, id)` (donde `id` es la PK de ventas).

5.3 Detalle de Índices Físicos

Tabla	Índice	Tipo	Columnas	Estructura Física
usuarios	PRIMARY	Clustered	id	B+ Tree, datos en hojas
usuarios	uk_email	Unique	email	B+ Tree, almacena (email, id)
productos	PRIMARY	Clustered	id	B+ Tree, datos en hojas
productos	idx_categoria	Secondary	categoria	B+ Tree, almacena (categoria, id)
ventas	PRIMARY	Clustered	id	B+ Tree, datos en hojas
ventas	idx_fecha	Secondary	fecha	B+ Tree, almacena (fecha, id)
detalle_ventas	PRIMARY	Clustered	id	B+ Tree, datos en hojas
detalle_ventas	idx_venta	Secondary	venta_id	B+ Tree, almacena (venta_id, id)

Tabla	Índice	Tipo	Columnas	Estructura Física
detalle_ventas	idx_producto	Secondary	producto_id	B+ Tree, almacena (producto_id, id)

---

6. Estrategia de Particionamiento Físico

Para tablas con alto volumen de inserciones (crecimiento histórico), se recomienda particionamiento por rango de fechas, que divide físicamente los datos en archivos separados:

6.1 Tabla `movimientos_stock`

- Particionamiento por `RANGE` en la columna `fecha` (tipo DATETIME)
- Cada partición almacena datos de un trimestre calendario
- Ejemplo de implementación física:

```sql
CREATE TABLE movimientos_stock (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto_id INT NOT NULL,
    tipo ENUM('entrada','salida','ajuste') NOT NULL,
    cantidad INT NOT NULL,
    stock_anterior INT NOT NULL,
    stock_nuevo INT NOT NULL,
    fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    usuario_id INT NULL,
    motivo VARCHAR(255) NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (producto_id) REFERENCES productos(id) ON DELETE
RESTRICT,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE SET NULL
) ENGINE=InnoDB
PARTITION BY RANGE (YEAR(fecha)*100 + MONTH(fecha)) (
    PARTITION p202601 VALUES LESS THAN (202602),
    PARTITION p202602 VALUES LESS THAN (202603),
    PARTITION p202603 VALUES LESS THAN (202604),
    PARTITION p_max VALUES LESS THAN MAXVALUE
);
```

- Cada partición genera un archivo `.ibd` separado: `movimientos_stock#P#p202601.ibd`

6.2 Tabla `sesiones_cyber`

- Particionamiento por `RANGE` en `hora_inicio` para sesiones antiguas
- Las particiones de sesiones cerradas se pueden archivar o mover a almacenamiento en frío

7. Parámetros de Configuración MySQL (Physical Tuning)

Ajustes recomendados en `my.ini` (Windows) para optimizar el rendimiento físico:

```
[mysqld]
# InnoDB Buffer Pool: 70-80% de RAM disponible (ej. 4GB para servidor con
6GB RAM)
innodb_buffer_pool_size = 4G
# Tamaño de los redo logs (256MB para cargas de escritura moderadas)
innodb_log_file_size = 256M
# Número de archivos de log (2 por defecto)
innodb_log_files_in_group = 2
# Método de flush de datos (O_DIRECT para evitar doble cacheo en Windows)
innodb_flush_method = O_DIRECT
# Tamaño de página (16KB por defecto, no cambiar a menos que se sepa lo que
se hace)
innodb_page_size = 16K
# Habilitar recolección de estadísticas para índices
innodb_stats_persistent = ON
```

---

8. Respaldo y Recuperación Física

8.1 Respaldo Físico (Hot Backup)

Se recomienda **Percona XtraBackup** para respaldos físicos en caliente (sin detener el servicio):

- Copia los archivos `.ibd`, redo logs y archivos de sistema
- Permite recuperación rápida (solo copiar archivos de vuelta al directorio de datos)

8.2 Respaldo Lógico (Complementario)

`mysqldump` para respaldos lógicos (SQL), útil para migraciones:

```
mysqldump -u root -p zwl > zwl_backup_2026-05-01.sql
```

8.3 Recuperación ante Fallos

InnoDB usa los redo logs (`ib_logfile*`) para recuperar transacciones confirmadas no volcadas al disco tras un reinicio inesperado.

---

9. Seguridad a Nivel Físico

1. **Permisos de archivos**: El directorio de datos de MySQL debe tener permisos restringidos (solo la cuenta del servicio MySQL y administradores)
2. **Encriptación en reposo**: Habilitar encriptación de tablespaces InnoDB (MySQL 8.0+):

   ```
   ALTER TABLE usuarios ENCRYPTION = 'Y';
   ```

3. **Binary logs**: Encriptar registros binarios para auditoría:

   ```
   [mysqld]
   binlog_encryption = ON
   ```

---

10. Monitoreo de Métricas Físicas

Métricas clave para supervisar el rendimiento del diseño físico:

- **Buffer pool hit rate**: Debe ser > 99% (consultar: `SHOW ENGINE INNODB STATUS;`)
- **Disk I/O**: Latencia de lectura/escritura en el directorio de datos
- **Índice usage**: Identificar índices no usados con `sys.schema_unused_indexes`

---

11. Conclusión

El diseño físico de ZWL está optimizado para el motor InnoDB en entornos Windows, con estructuras de almacenamiento eficientes, índices B+ Tree para acceso rápido y parámetros de configuración ajustados para cargas de trabajo transaccionales moderadas. La estrategia de particionamiento para tablas históricas garantiza que el rendimiento no se degrade con el tiempo.